Improvement Plan

> Every item below was found by reading the actual code, scripts and manifests — not from a generic checklist. Each has the evidence, the impact, and a concrete fix.
>
> **Note on Floci:** most of these produce **no visible symptom** locally — pods don't restart, AZs don't fail, IAM isn't enforced. They matter on real AWS, and in interviews.

Summary

#	Issue	Tier	Effort
1	ALB targets never re-sync — pod restart ⇒ 503	🔴 breaks	M
2	Single NAT Gateway — AZ-a failure kills AZ-b's internet	🔴 breaks	S
3	No NetworkPolicy — any pod can reach any database	🟠 security	M
4	Data subnets have internet egress (exfiltration path)	🟠 security	S
5	NACL association command passes the wrong ID type	🟠 security	S
6	~1 test file per service, no Testcontainers	🟢 **career**	L
7	No correlation/trace ID across the saga	🟢 **career**	M
8	Plain-text logs — Loki can't filter fields	🟢 **career**	S
9	`nodeSelector` unused — node group labels do nothing	🟣 consistency	S
10	Data tier + `db-sg` created but empty/unattached	🟣 consistency	S
11	Thin bean validation (17 annotations across 6 services)	🟣 quality	S

Recommended order for a Java developer: 6 → 7 → 8, then 1 → 3, then the rest. Tier 3 is weakest *and* most interviewed-on.

🔴 Tier 1 — Breaks on real AWS

1. ALB targets never re-sync — ✅ FIXED

> **Resolved** via `TargetGroupBinding` rather than the full Ingress migration originally proposed below. See **"How this was actually fixed"** at the end of this item. The analysis is kept because it explains *why* the fix was needed.

Evidence — `scripts/05-deploy.sh:70-80`:

```
KONG_IPS=$(kubectl get pods -n ecommerce -l app=kong -o jsonpath='{.items[*].status.podIP}')
for ip in $KONG_IPS; do
  aws elbv2 register-targets --target-group-arn $TG_ARN --targets Id=$ip,Port=8000
done
```

This is a **one-time snapshot** taken at deploy.

What breaks:

Event	Result
Kong pod restarts → new IP	ALB still sends to the **dead IP** → 503
HPA scales up	new pods get **no traffic**
Pod deleted	its IP sits **unhealthy forever** in the target group

Also conflicting: `k8s/kong/kong.yaml` has `type: LoadBalancer` plus `service.beta.kubernetes.io/aws-load-balancer-type: alb`. Those annotations are instructions for the AWS Load Balancer Controller — which is **not installed** (`grep` finds it nowhere). So they do nothing today, and if the controller *were* installed they'd create a **second** ALB alongside the manually-built one.

Fix — let the controller own the ALB:

1. Install the AWS Load Balancer Controller (IAM policy → OIDC provider → IRSA service account → controller manifest). This is exactly the Udemy course's Lecture 68 procedure.
2. Delete the manual `register-targets` block from `05-deploy.sh`.
3. Delete ALB + target group + listener creation from `02-alb.sh` — the controller creates them from the Ingress.
4. Change Kong's Service to `type: ClusterIP` and drop the `aws-load-balancer-*` annotations.
5. Add an Ingress:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ecommerce-ingress
  namespace: ecommerce
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'
    alb.ingress.kubernetes.io/ssl-redirect: '443'
    alb.ingress.kubernetes.io/certificate-arn: <ACM_CERT_ARN>
    alb.ingress.kubernetes.io/healthcheck-path: /status
    alb.ingress.kubernetes.io/healthcheck-port: '8100'
    alb.ingress.kubernetes.io/wafv2-acl-arn: <WAF_ACL_ARN>
spec:
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: kong-proxy
                port:
                  number: 8000
```

The controller then watches Endpoints and keeps the target group in sync automatically — which is the whole point.

> ⚠️ **Floci caveat:** the controller needs real IAM/OIDC/IRSA, which Floci doesn't faithfully emulate. Expect this to work on real AWS and be partially broken locally.

✅ How this was actually fixed — `TargetGroupBinding`

The migration above hands the **whole ALB** to the controller. That works, but it throws away everything `02-alb.sh` already builds correctly — WAF association, access logs, deletion protection, TLS policy, the 301 redirect.

There's a narrower option, and it's what was implemented: **keep owning the ALB, and delegate only the target list.**

New file — `k8s/alb/targetgroupbinding.yaml`:

```
apiVersion: elbv2.k8s.aws/v1beta1
kind: TargetGroupBinding
metadata:
  name: kong-proxy-tgb
  namespace: ecommerce
spec:
  serviceRef:
    name: kong-proxy
    # The SERVICE port, not the container port. This said 8000 — the container
    # port — and kong-proxy publishes 80/443. The controller accepted the object
    # and logged "Successfully reconciled", while beside it sat
    #   BackendNotFound: unable to find port 8000 on service ecommerce/kong-proxy
    # and the target group stayed empty. Only caught by registering it against a
    # real ALB and looking at the targets.
    port: 80
  targetGroupARN: TG_ARN_PLACEHOLDER      # from scripts/.env
  targetType: ip
  networking:
    ingress:
      - from:
          - securityGroup:
              groupID: ALB_SG_PLACEHOLDER
        ports:
          - protocol: TCP
            port: 8000
```

`scripts/05-deploy.sh` step 6 — the manual snapshot loop was replaced with:

```
# fail early if the controller isn't installed — it provides the CRD
if ! kubectl get crd targetgroupbindings.elbv2.k8s.aws >/dev/null 2>&1; then
  echo "  ❌ AWS Load Balancer Controller not found."
  exit 1
fi

sed "s|TG_ARN_PLACEHOLDER|$TG_ARN|g; s|ALB_SG_PLACEHOLDER|$ALB_SG|g" \
  k8s/alb/targetgroupbinding.yaml | kubectl apply -f -
```

Both `$TG_ARN` and `$ALB_SG` already exist in `scripts/.env` (written by `02-alb.sh` and `01-vpc.sh`), so nothing else changes.

What this changes:

	Before	After
ALB / listeners / rules	`02-alb.sh`	`02-alb.sh` — **unchanged**
Target registration	one snapshot at deploy	**controller, continuously**
Pod restart	503s until manual re-run	handled in ~2s
HPA scale-up	new pods get no traffic	registered automatically

Still required: the AWS Load Balancer Controller must be installed — it supplies the `TargetGroupBinding` CRD. The script now checks for it and exits with instructions rather than failing obscurely.

Why this over full Ingress mode: the ALB carries WAF, access logs, deletion protection and a TLS-1.3 policy configured in `02-alb.sh`. Reproducing all of that through Ingress annotations is possible but fiddly, and it moves control of a security-relevant resource into a controller. Delegating only target membership fixes the actual bug and leaves the rest alone.

---

2. Single NAT Gateway

Evidence — `scripts/01-vpc.sh:56, 61-64` :

```
NAT=$(aws ec2 create-nat-gateway --subnet-id $PUB1 ...)   # AZ-a only
associate-route-table --route-table-id $RT_PRIV --subnet-id $PRIV1   # AZ-a
associate-route-table --route-table-id $RT_PRIV --subnet-id $PRIV2   # AZ-b → crosses AZ
```

What breaks: if AZ-a fails, nodes in **AZ-b lose internet too** — no image pulls, no SES. Everything else was deliberately built across two AZs to survive exactly this; the single NAT undoes it. Also adds cross-AZ data-transfer charges on all AZ-b egress.

Fix — one NAT per AZ, one private route table per AZ:

```
EIP1=$(aws ec2 allocate-address --domain vpc --query AllocationId --output text)
EIP2=$(aws ec2 allocate-address --domain vpc --query AllocationId --output text)
NAT1=$(aws ec2 create-nat-gateway --subnet-id $PUB1 --allocation-id $EIP1 ...)
NAT2=$(aws ec2 create-nat-gateway --subnet-id $PUB2 --allocation-id $EIP2 ...)

RT_PRIV1=$(aws ec2 create-route-table --vpc-id $VPC_ID ...)
aws ec2 create-route --route-table-id $RT_PRIV1 --destination-cidr-block 0.0.0.0/0 --nat-gateway-id $NAT1
aws ec2 associate-route-table --route-table-id $RT_PRIV1 --subnet-id $PRIV1

RT_PRIV2=$(aws ec2 create-route-table --vpc-id $VPC_ID ...)
aws ec2 create-route --route-table-id $RT_PRIV2 --destination-cidr-block 0.0.0.0/0 --nat-gateway-id $NAT2
aws ec2 associate-route-table --route-table-id $RT_PRIV2 --subnet-id $PRIV2
```

Cost: ~$32/month per NAT. This is the standard production trade-off.

🟠 Tier 2 — Security

3. No NetworkPolicy — flat pod network

Evidence: `grep -rl 'kind: NetworkPolicy' k8s/` → **nothing**.

Kubernetes defaults to **every pod can reach every pod on every port**. So if `notification-service` is compromised (a dependency CVE, say), the attacker immediately reaches all 5 Postgres instances, Redis, and Kafka — none of which `notification-service` legitimately needs.

Only the **passwords** stand in the way, and those are env vars on pods in the same cluster.

Why `db-sg` can't help: security groups filter at the ENI level; pod-to-pod traffic rides the CNI overlay and largely never traverses them. The tier boundary has to be re-created at the Kubernetes layer, where the workloads actually live.

Fix — default deny, then explicit allows:

```
# 1. Deny everything in the namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
```

```
  namespace: ecommerce
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
# 2. Only order-service may reach postgres-order
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-order-to-its-db
  namespace: ecommerce
spec:
  podSelector:
    matchLabels:
      app: postgres-order
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: order-service
      ports:
        - protocol: TCP
          port: 5432
```

Repeat per service→DB pair. Also allow DNS egress to `kube-system`, and service→Kafka / service→Redis as needed.

> ⚠️ Requires a CNI that enforces NetworkPolicy (Calico, Cilium). The AWS VPC CNI needs the network-policy agent enabled.
>
> ✅ **k3s does enforce NetworkPolicy** — verified by live test on the Floci cluster: a busybox pod was blocked from MySQL, MongoDB and the internet while the application kept working. k3s pairs Flannel with an embedded kube-router policy controller. Plain upstream Flannel on its own does not enforce policy — so the rule is **verify enforcement after applying**, not "assume it works" or "assume it doesn't".

4. Data subnets have internet egress

Evidence — `scripts/01-vpc.sh:63-64` puts `DATA1` / `DATA2` on the **private** route table, which carries `0.0.0.0/0 → NAT`.

Risk: inbound is still impossible (NAT is one-way), but a compromised database can **send data out** — exfiltration — or pull down tooling.

Fix:

```
RT_DATA=$(aws ec2 create-route-table --vpc-id $VPC_ID --query 'RouteTable.RouteTableId' --output text)
# add NO 0.0.0.0/0 route — the implicit `local` route is all it needs
aws ec2 associate-route-table --route-table-id $RT_DATA --subnet-id $DATA1
aws ec2 associate-route-table --route-table-id $RT_DATA --subnet-id $DATA2
```

Then add **VPC endpoints** for whatever AWS services the tier genuinely needs (S3 Gateway endpoint for backups; Interface endpoints for Secrets Manager, KMS, CloudWatch Logs). Specific destinations, never `0.0.0.0/0`.

> Only meaningful once something actually runs in the data tier — see item 10.

5. NACL association passes the wrong ID type

Evidence — `scripts/01-vpc.sh:176-179`:

```
aws ec2 replace-network-acl-association --network-acl-id $NACL --association-id $PUB1
```

`--association-id` expects `aclassoc-xxxxx`; `$PUB1` holds `subnet-xxxxx`. Real AWS rejects this, and `set -e` would abort the script.

Fix — look up the current association first:

```
for SUBNET in $PUB1 $PUB2; do
  ASSOC=$(aws ec2 describe-network-acls \
    --filters Name=association.subnet-id,Values=$SUBNET \
    --query "NetworkAcls[0].Associations[?SubnetId=='$SUBNET'].NetworkAclAssociationId" \
    --output text)
  aws ec2 replace-network-acl-association --network-acl-id $NACL --association-id $ASSOC
done
```

🟢 Tier 3 — Highest value for a Java developer

6. Testing depth

Evidence:

```
user / product / order / inventory / payment / notification  →  1 test file each
Testcontainers in any pom.xml                                →  0
```

The CI pipeline advertises a `test ×6 parallel matrix`, but there's very little running inside it.

Why this matters most: testing is among the most heavily probed areas in Java interviews — and the questions land squarely on this project's best ideas:

- "*How do you test a Kafka consumer?*"
- "*How do you test the saga's compensation path?*"
- "*How do you prove the Redis lock prevents oversell?*"

Fix — add Testcontainers:

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
```

```
  <groupId>org.testcontainers</groupId>
  <artifactId>kafka</artifactId>
  <scope>test</scope>
</dependency>
```

```
@SpringBootTest
@Testcontainers
class OrderSagaIT {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15");

    @Container
    static KafkaContainer kafka = new KafkaContainer(
        DockerImageName.parse("confluentinc/cp-kafka:7.5.0"));

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", postgres::getJdbcUrl);
        r.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
    }

    @Test
    void paymentFailure_releasesReservedStock() {
        // place order → assert stock reserved
        // emit payment.processed(success=false)
        // assert order CANCELLED *and* stock released
    }
}
```

The three tests worth writing first — they target this project's actual design:

1. **Compensation path** — payment fails ⇒ order `CANCELLED` **and** stock released
2. **Redis lock under concurrency** — two threads, one unit of stock, exactly one wins
3. **Idempotency** — same `order.created` delivered twice ⇒ stock decremented **once**

7. No correlation / trace ID

Evidence: no `micrometer-tracing`, `sleuth`, `opentelemetry`, or `zipkin` in any `pom.xml`.

The problem: an order fails after touching `order-service` → Kafka → `inventory-service` → Kafka → `payment-service` → `notification-service`. Debugging means opening **five** services' logs and correlating by timestamp, hoping no other order ran concurrently.

Harder here than usual: the flow is asynchronous over Kafka, so the trace ID must travel in **Kafka message headers**, not HTTP headers.

Fix:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>
```

Spring Kafka propagates tracing headers automatically once tracing is on the classpath. Then put the ID into MDC so **every** log line carries it:

```
MDC.put("traceId", tracer.currentSpan().context().traceId());
```

Result: one filter in Loki (`| json | traceId="a3f9c2"`) shows the entire order journey across all six services, in order.

8. Plain-text logs

Evidence: no `logback-spring.xml` or `log4j2.xml` anywhere → Spring Boot's default text format. Loki, Prometheus and Grafana **are** deployed (`k8s/monitoring/monitoring.yaml`), so the infrastructure is ready — the log format isn't.

Today:

```
2026-07-26 15:23:01 INFO OrderService - Order created: 1234
```

Loki stores it but can only substring-match.

Fix — `src/main/resources/logback-spring.xml` :

```
<configuration>
  <appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <includeMdcKeyName>traceId</includeMdcKeyName>
      <customFields>{"service":"${SPRING_APPLICATION_NAME:-unknown}"}</customFields>
    </encoder>
  </appender>
  <root level="INFO">
    <appender-ref ref="JSON"/>
  </root>
</configuration>
```

```
<dependency>
  <groupId>net.logstash.logback</groupId>
  <artifactId>logstash-logback-encoder</artifactId>
</dependency>
```

Then:

```
{"ts":"...","level":"INFO","service":"order-service","traceId":"a3f9c2","msg":"Order created","orderId":1234}
```

...and `{service="order-service"} | json | orderId=1234` works.

Also: ~8 log statements per service is thin for a saga. Log **every state transition** (`PENDING → STOCK_RESERVED → CONFIRMED`) with the order ID, so the flow can be reconstructed from logs alone.

Tier 4 — Consistency

9. `nodeSelector` unused

Evidence: node groups are labelled `role=general` / `role=high-memory` (`04-eks.sh`), but `grep -rn 'nodeSelector' k8s/` finds **nothing**.

So the scheduler places pods anywhere — Kafka may well be running on the cheap `m5.large` while `user-service` sits on the expensive `r5.large` . The two-nodegroup split currently buys nothing.

Fix — on `inventory` , `payment` , `notification` , `kafka` :

```
spec:
  template:
    spec:
      nodeSelector:
        role: high-memory
```

...and `role: general` on Kong, user, product, order.

10. Data tier exists on paper only

Evidence:

- `data-az1` / `data-az2` created — **nothing deployed into them**
- `db-sg` created with 3 rules — **never attached to anything** (referenced again only in `06-teardown.sh`, to delete it)
- Postgres ×5, Redis, Kafka are **StatefulSets**, so they run as pods on EKS nodes in the **private** subnets

The documented 3-tier design is really a **2-tier deployment**.

Pick one — don't leave it ambiguous:

Option	Do this
A — make it real	Replace the StatefulSets with **RDS / ElastiCache / MSK**, placed in the data subnets via subnet groups. `db-sg` and item 4 then genuinely apply. Closest to production; costs money.
B — accept 2-tier	Delete the data subnets and `db-sg`, update the docs, and enforce isolation with **NetworkPolicy** (item 3) instead. Honest and simpler.

Either is defensible. Leaving it as-is means the docs claim a boundary the deployment doesn't have — and that's the version that looks bad in a review.

11. Thin input validation

Evidence: 17 Bean Validation annotations total across all six services (`@NotBlank` ×7, `@NotNull` ×3, `@Valid` ×3, `@Email` ×2, `@Size` ×1, `@Min` ×1).

Note: this is *not* a SQL-injection risk — all queries are properly parameterized (`:productId` , `:qty` with `@Param`), and Spring Data JPA parameterizes derived queries automatically. That part is already correct.

It's about rejecting bad data early with clear errors:

```
public record CreateOrderRequest(
    @NotNull @Positive Long productId,
    @NotNull @Min(1) @Max(100) Integer quantity,
    @NotBlank @Email String email
) {}
```

Also worth doing: give the app's DB user only `SELECT, INSERT, UPDATE, DELETE` — not `DROP TABLE` or `CREATE USER` — so that even a hypothetical injection can't destroy schema.

What's already right

Worth stating, so the list above isn't read as "the project is bad":

Parameterized queries everywhere	✅ no SQL-injection exposure
`POSTGRES_PASSWORD` + Redis `--requirepass` from Secrets	✅ most demos skip Redis auth
SLF4J, **zero** `System.out.println`	✅
Prometheus + Grafana + **Loki** deployed	✅ ahead of most projects
WAF: OWASP + rate limit + SQLi	✅
TLS 1.3 only, HTTP→HTTPS 301	✅
ECR `scanOnPush` + keep-10 lifecycle	✅
Image tag = git SHA (not `:latest`)	✅ rollback-safe
ALB: deletion protection, access logs, 30s deregistration delay	✅
Saga with **compensating transactions**	✅ genuine distributed-systems design
DB-per-service	✅ real boundaries
CI: parallel matrix → Trivy → k3d → saga smoke test → approval gate	✅

The architecture is strong. The gaps are in operational depth (tests, tracing) and in a few places where intent and deployment drifted apart.

PART 2 — Floci vs Real AWS: full compatibility scan

> Every script and manifest read line by line. The question answered here: **"if I run this exact code against real AWS tomorrow, what happens?"**
>
> Short version: **the infrastructure builds, then every pod fails to start.**

Verdict at a glance

Category	Count
🔴 Works on Floci → **hard-fails or misbehaves on AWS**	9
🟡 Works on AWS → **fails/degrades on Floci**	4
💰 Silent **cost leaks** on AWS	1 (expensive)
🟢 Works identically on both	most of the Kubernetes layer

🔴 A. Works on Floci — HARD FAILS on real AWS

A1. `REGISTRY=localhost:5100` — every single pod fails to start

Evidence — `scripts/03-ecr.sh:20`, used in every image reference:

```
REGISTRY=localhost:5100
```

```
image: REGISTRY/ecommerce/user-service:IMAGE_TAG
  → becomes → localhost:5100/ecommerce/user-service:a3f9c21
```

On Floci: works — `localhost:5100` is the ECR-registry container on your machine.

On AWS: `localhost` inside a pod means **that node itself**. EKS nodes have no registry on port 5100. Result: `ImagePullBackOff` **on all 6 microservices.** Postgres/Redis/Kafka/Kong still start (public Docker Hub images), so you get a half-running cluster where every one of *your* services is dead.

Fix:

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
REGISTRY=$ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com
```

Real ECR also requires the nodes' IAM role to carry `AmazonEC2ContainerRegistryReadOnly` — `04-eks.sh` already attaches it ✅.

A2. No StorageClass + no EBS CSI driver — the deploy never gets past step 3

Evidence: `grep -rn storageClassName k8s/` → **nothing**. `grep -rn "ebs-csi\|create-addon" scripts/` → **nothing**.

Seven `volumeClaimTemplates` (5 Postgres + Redis + Kafka, 2-20 Gi) rely on the cluster's *default* StorageClass.

	Default StorageClass?
Floci / k3s	✅ `local-path`, built in
Real EKS	❌ **none** — the EBS CSI driver is an add-on you must install

On AWS: every PVC sits `Pending` forever → StatefulSets never become Ready → `kubectl wait --timeout=180s` (line 41) fails → `set -e` **aborts the script**. Nothing after step 3 ever runs.

Fix:

```
eksctl create addon --name aws-ebs-csi-driver --cluster ecommerce-cluster \
  --service-account-role-arn <EBS_CSI_ROLE_ARN> --force
```

...plus a StorageClass, and set `storageClassName: gp3` explicitly on every `volumeClaimTemplate` rather than relying on a default.

> Also worth knowing: `local-path` on Floci is **node-local**. A pod rescheduled to another node loses its data. Real EBS follows the pod within its AZ.

A3. ACM certificate for `.local` — never validates

Evidence — `scripts/02-alb.sh:30,42`:

```
DOMAIN="ecommerce.local"
aws acm request-certificate --domain-name $DOMAIN --validation-method DNS
```

`.local` is **not a real public TLD** (it's reserved for mDNS). A public CA cannot issue for it. The certificate stays `PENDING_VALIDATION` **forever**, so the HTTPS listener at line 284 fails — and with it the ALB's entire 443 path.

Fix: use a domain you actually own, or terminate TLS elsewhere for demos.

A4. NACL association passes the wrong ID type

Already documented as item 5 above — `--association-id $PUB1` passes `subnet-xxxxx` where `aclassoc-xxxxx` is required. Floci accepts it; **real AWS rejects it and `set -e` aborts the script.**

A5. Hardcoded ELB hosted-zone ID

Evidence — `scripts/02-alb.sh:348`:

```
"AliasTarget": { "HostedZoneId": "Z35SXDOTRQ7X7K", ... }
```

This is **not your hosted zone** — it's AWS's fixed per-region constant for ELB, and it differs by region. This value does not look like the `ap-south-1` one. Wrong value ⇒ the alias record is rejected or points nowhere.

Fix — look it up instead of hardcoding:

```
ELB_ZONE=$(aws elbv2 describe-load-balancers --load-balancer-arns $ALB_ARN \
  --query 'LoadBalancers[0].CanonicalHostedZoneId' --output text)
```

A6. HPA with no metrics-server — never scales

Evidence: HPAs exist in `k8s/kong/kong.yaml` and `k8s/services/deployments.yaml`. `grep -rl metrics-server k8s/ scripts/` → **nothing**.

	metrics-server
Floci / k3s	✅ bundled
Real EKS	❌ **not installed by default**

On AWS: `kubectl get hpa` shows `<unknown>/70%` and the replica count never moves. Silent — no error, no event, it just does nothing. Under load you get outages while HPA sits idle.

Fix:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

Also required: `resources.requests.cpu` on every container, or HPA has no denominator to compute a percentage against.

A7. Subnets carry no `kubernetes.io/role` tags — the ALB controller can't find them

Evidence — `scripts/01-vpc.sh:37-42` tags subnets with **only** a `Name`:

```
aws ec2 create-tags --resources $PUB1  --tags Key=Name,Value=public-az1
aws ec2 create-tags --resources $PRIV1 --tags Key=Name,Value=private-az1
```

`grep -rn "kubernetes.io/role" scripts/` → **nothing**.

Why this matters: the AWS Load Balancer Controller **auto-discovers** which subnets to place an ALB in — by reading tags. No tags, no discovery:

```
Error: couldn't auto-discover subnets: unable to resolve at least one subnet
```

Required tags:

Subnet	Tag	Value
public	`kubernetes.io/role/elb`	`1`
private	`kubernetes.io/role/internal-elb`	`1`
both (older EKS / shared VPCs)	`kubernetes.io/cluster/ecommerce-cluster`	`shared`

On Floci this never surfaced — the ALB was built manually with explicit `--subnets $PUB1 $PUB2`, so nothing ever needed to auto-discover anything. The moment you adopt the controller (the fix for item 1), this becomes a blocker.

Fix:

```
aws ec2 create-tags --resources $PUB1 $PUB2 \
  --tags Key=kubernetes.io/role/elb,Value=1 \
         Key=kubernetes.io/cluster/ecommerce-cluster,Value=shared
aws ec2 create-tags --resources $PRIV1 $PRIV2 \
  --tags Key=kubernetes.io/role/internal-elb,Value=1 \
         Key=kubernetes.io/cluster/ecommerce-cluster,Value=shared
```

Data subnets need **no** tags — nothing should ever place a load balancer there.

A8. Data subnets: allocated, routed, tagged — and completely empty

Evidence:

- `data-az1` (`10.0.5.0/24`) and `data-az2` (`10.0.6.0/24`) are created and tagged
- `db-sg` is created with rules for 5432 / 6379 / 9092
- **Nothing is ever deployed into them.** Postgres ×5, Redis and Kafka are StatefulSets, so they run as pods on EKS nodes — in the **private** subnets
- `db-sg` is never attached to anything; after `01-vpc.sh` the only other mention is `06-teardown.sh:33` , deleting it

On Floci: invisible. Nothing enforces placement, so no symptom.

On AWS: not a crash — but three real consequences:

1. **512 IP addresses reserved for nothing** across the two subnets
2. **The documented security boundary doesn't exist.** `ARCHITECTURE.md` claims a 3-tier network where databases are unreachable except from the app tier. In reality the databases sit on the same nodes, in the same subnet, under the same `eks-sg` — and with no NetworkPolicy (item 3), *every* pod can reach *every* database. **The diagram is more secure than the deployment.**
3. `db-sg` **'s rules are dead config** — someone will eventually "fix" a connectivity problem by editing a security group that was never in the path.

This is the single biggest gap between what the docs claim and what runs. See Tier 4 item 10 for the two ways to close it (managed services, or commit to 2-tier plus NetworkPolicy).

A9. `sed -i` rewrites a git-tracked file

Evidence — `scripts/05-deploy.sh:31` :

`sed -i "s|REGISTRY|$REGISTRY|g" k8s/services/configmap.yaml`

This edits the repo file **in place**. After one run the `REGISTRY` placeholder is gone permanently, your working tree is dirty, and deploying to a different registry silently uses the old value.

Note line 55 does it correctly — pipes to `kubectl` without touching the file:

`sed "s|REGISTRY|$REGISTRY|g; s|IMAGE_TAG|$IMAGE_TAG|g" k8s/services/deployments.yaml | kubectl apply -f -`

Fix: make line 31 match line 55's pattern.

🟡 B. Works on AWS — fails or degrades on Floci

B1. `target-type: ip` needs the AWS VPC CNI

Pods must hold **real VPC IPs** for the ALB to reach them directly.

CNI	Pod IPs	`target-type: ip`
AWS VPC CNI (EKS default)	`10.0.3.x` — real VPC	✅
k3s / Flannel (Floci)	`10.42.x.x` **— overlay**	❌ unroutable from the ALB

On Floci you'd need `target-type: instance` (ALB → NodePort → kube-proxy → pod).

B2. AWS Load Balancer Controller needs real IRSA

The fix for the flagship issue (item 1) depends on IAM policy + OIDC provider + IRSA — none of which Floci faithfully emulates. The commands succeed; the trust relationship isn't genuinely enforced. Expect it to work on AWS and be partially broken locally.

B3. NetworkPolicy needs an enforcing CNI

✅ **Corrected by live testing.** k3s **does** enforce NetworkPolicy — it pairs Flannel with an embedded kube-router policy controller. Verified on the Floci cluster: after applying a default-deny plus targeted allows, a busybox pod was blocked from MySQL, MongoDB and the internet, while all application endpoints kept returning 200.

Still true elsewhere: plain upstream Flannel does not enforce policy, and on EKS the AWS VPC CNI needs its network-policy agent enabled (or Calico/Cilium). **Always verify enforcement after applying** rather than assuming either way.

B4. IAM is not enforced on Floci

`04-eks.sh` creates roles and attaches policies, and Floci accepts them — but never checks them. So a missing permission that would be a hard `AccessDenied` on AWS passes silently here. **Floci cannot validate your IAM setup.**

💰 C. Cost leak — teardown leaves billable resources running

Evidence: `06-teardown.sh` deletes the nodegroups, cluster, listeners, target group, ALB (correctly disabling deletion protection first ✅) and security groups.

It **never** deletes:

Orphaned resource	Approx. monthly cost
NAT Gateway	**~$32** + data processing
Elastic IP	~$3.60
S3 buckets (flow logs + ALB logs)	storage + request costs
VPC, 6 subnets, IGW, route tables, NACL	free, but clutter
ACM certificate, Route 53 hosted zone	~$0.50/zone

You'd run "teardown", believe everything is gone, and keep paying **~$36/month** indefinitely. On Floci this costs nothing, so the bug is invisible.

Fix — append to the teardown, in this order:

```
aws ec2 delete-nat-gateway --nat-gateway-id $NAT
aws ec2 wait nat-gateway-deleted --nat-gateway-ids $NAT   # must finish before releasing the EIP
aws ec2 release-address --allocation-id $EIP
aws ec2 delete-subnet --subnet-id $PUB1   # ...and the other five
aws ec2 delete-route-table --route-table-id $RT_PUB
aws ec2 detach-internet-gateway --internet-gateway-id $IGW_ID --vpc-id $VPC_ID
aws ec2 delete-internet-gateway --internet-gateway-id $IGW_ID
aws ec2 delete-vpc --vpc-id $VPC_ID
aws s3 rb s3://$LOG_BUCKET --force
aws s3 rb s3://$FL_BUCKET --force
```

Order matters — a NAT Gateway must be fully deleted before its EIP can be released, and a VPC won't delete while subnets still exist.

🟢 D. Behaves the same on both

- All Kubernetes objects: Deployments, StatefulSets, Services, ConfigMaps, Secrets, namespaces
- Postgres, Redis, Kafka as containers (identical images)
- Kong's declarative config and its routing to `*.svc.cluster.local` names
- The Kafka saga, Redis locking, JWT/session logic — pure application behaviour
- `kubectl` workflows, rollouts, probes

This is the reassuring part: the *application* layer is genuinely portable. Every failure above is infrastructure wiring, not your code.

⏱️ E. Operational differences worth expecting

Operation	Floci	Real AWS
`create-cluster` → ACTIVE	seconds	**10-15 min**
Nodegroup → ACTIVE	seconds	~5 min
NAT Gateway → Available	instant	~2 min
ACM validation	instant	~5 min (with a real domain)
ALB provisioning	instant	~3 min
Full stack build	~2 min	**~30 min**

`aws eks wait cluster-active` is already used ✅. Add similar waits for the NAT Gateway and ALB, or later steps will race ahead of resources that aren't ready.

Priority order to make this AWS-ready

#	Fix	Without it
1	**A1** — real ECR registry URL	every microservice `ImagePullBackOff`
2	**A2** — EBS CSI addon + StorageClass	deploy aborts at step 3
3	**A4** — NACL association lookup	`01-vpc.sh` aborts
4	**A3** — a real domain for ACM	no HTTPS listener
5	**A7** — `kubernetes.io/role` subnet tags	ALB controller can't find subnets
6	**A5** — look up the ELB zone ID	DNS record broken
7	**A6** — metrics-server	HPA silently dead
8	**C** — teardown the NAT + EIP	~$36/month forever
9	**A8** — resolve the data-tier fiction	docs claim security that doesn't exist
10	**A9** — stop `sed -i` on a tracked file	repo drift

1-3 are hard blockers — the stack does not come up at all without them. **5 becomes a blocker** the moment you adopt the ALB controller (Tier 1 item 1), since it depends on subnet auto-discovery.

Everything in the earlier Tier 1/2 list (ALB target sync, NAT per AZ, NetworkPolicy) comes *after* these, because those assume a running deployment.

The subnet story, end to end

Worth reading as one thread, since it spans several findings:

Tier	Created	Routed	Tagged for EKS	Occupied	Secured by
public ×2	✅	✅ → IGW	❌ missing `role/elb`	ALB, NAT	`alb-sg` ✅
private ×2	✅	✅ → NAT	❌ missing `role/internal-elb`	**everything** — Kong, 6 services, 5 Postgres, Redis, Kafka	`eks-sg` ✅
data ×2	✅	⚠️ shares the private table (has NAT egress)	—	**nothing**	`db-sg` ❌ never attached

Three separate problems visible in one table:

1. **Public/private subnets lack the tags** the ALB controller needs (A7)
2. **Data subnets have internet egress** they shouldn't (Tier 2 item 4)
3. **Data subnets are empty and `db-sg` is orphaned** (A8) — so the 3-tier design exists only in the diagrams

Fixing #1 unblocks the controller. Fixing #2 and #3 together requires the decision in Tier 4 item 10: either move the datastores to managed services in the data subnets, or delete the data tier and enforce isolation with NetworkPolicy instead.